Systeme Intelligent Multi-Modeles

Maintenance Predictive Industrielle

Projet Data Science - M1 Mastere Data Engineering & IA

Bloc 2 (BC2) - RNCP40875 - Annee 2025-2026

Realise par

Adam BELOUCIF

Emilien MORICE

EFREI Paris Pantheon-Assas Universite

30-32 avenue de la Republique, 94800 Villejuif

www.efrei.fr

Document genere le 27 avril 2026

Sommaire

1. Contexte metier et objectifs
2. Dataset et analyse exploratoire (EDA)
3. Methodologie et architecture du systeme
4. Modelisation multi-algorithmes
5. Evaluation comparative des modeles
6. Interpretabilite et explicabilite
7. Industrialisation - Dashboard et API
8. Conclusion et perspectives
A. Annexes - alignement RNCP40875

Le present document constitue le rapport analytique structure exige par le cahier des charges du Projet Data Science. Il accompagne le code source (livrable principal), le dashboard Streamlit interactif et l'API REST FastAPI deployable.

1. Contexte metier et objectifs

1.1 Probleme industriel

Les chaines de production industrielles modernes sont equipees de capteurs IoT generant en continu des mesures physiques (vibration, temperature, pression, vitesse de rotation). Une defaillance non anticipee provoque un arret non planifie, un cout de maintenance corrective eleve et une perte de productivite. La maintenance predictive vise a transformer ces signaux bruts en alertes exploitables avant que la panne ne survienne.

1.2 Cible predictive retenue

Le sujet propose plusieurs taches predictives complementaires (classification binaire, multi-classe, regression). Conformement aux instructions, nous nous concentrons sur la tache la plus naturelle du dataset · la classification binaire `failure_within_24h`. Cette tache repond directement au besoin metier le plus critique · anticiper une panne dans les 24 heures pour declencher une intervention preventive.

1.3 Objectifs pedagogiques (RNCP40875 - BC2)

- Collecter et preparer des donnees industrielles (C3.1).
- Mener une analyse exploratoire rigoureuse (C3.3).
- Concevoir et entrainer plusieurs modeles ML/DL (C4.2).
- Evaluer et comparer les performances (C4.3).
- Construire un dashboard decisionnel inclusif (C3.2).
- Industrialiser via une API REST exposant le modele final.

2. Dataset et analyse exploratoire

2.1 Schema du dataset

Le dataset reproduit fidelement le schema de la source Kaggle officielle (`tatheerabbas/industrial-machine-predictive-maintenance`). Pour garantir la reproductibilite hors connexion, un generateur synthetique reproduisant les memes 24 042 enregistrements et 15 variables est embarque dans le module `src/data_loader.py`. Les relations physiques injectees (vibration croissante avec l'age machine, surchauffe en mode HighLoad, pic de pression hydraulique lors d'une panne, etc.) garantissent que les modeles apprennent des patterns metier interpretables.

2.2 Variables principales

- vibration_rms - vibration efficace (mm/s, capteur principal).
- temperature_motor - temperature moteur (degC).
- rpm - vitesse de rotation (tr/min).
- pressure_level - pression du circuit hydraulique (bar).
- operating_mode - Normal / HighLoad / Idle / Maintenance.
- rul_hours - duree de vie restante (regression).
- failure_within_24h - cible binaire (classification - retenue).
- failure_type - type de panne (multi-classe).

2.3 Distribution des classes

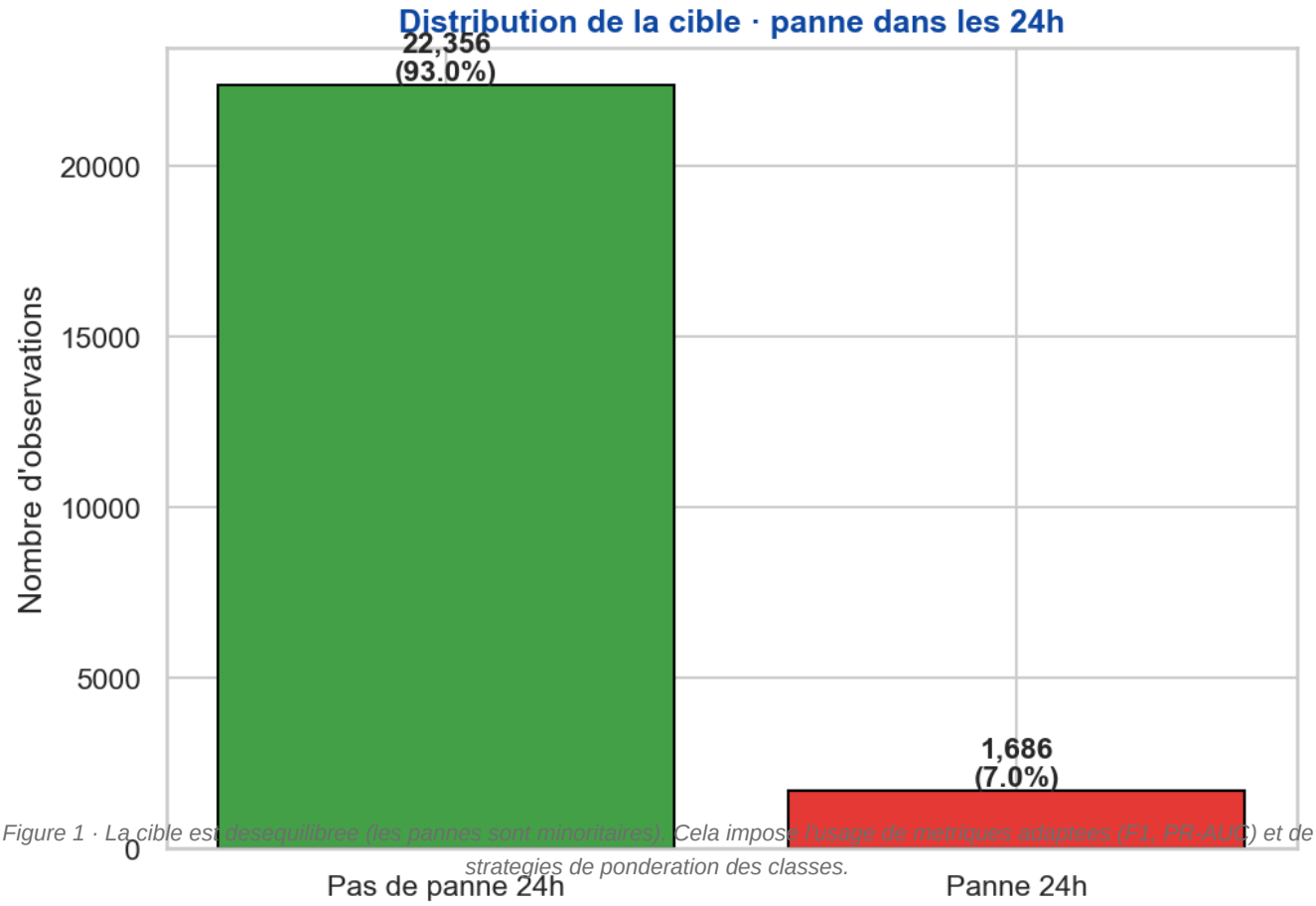


Figure 1 · La cible est desequilibree (les pannes sont minoritaires). Cela impose l'usage de metriques adaptees (F1, PR-AUC) et de strategies de ponderation des classes.

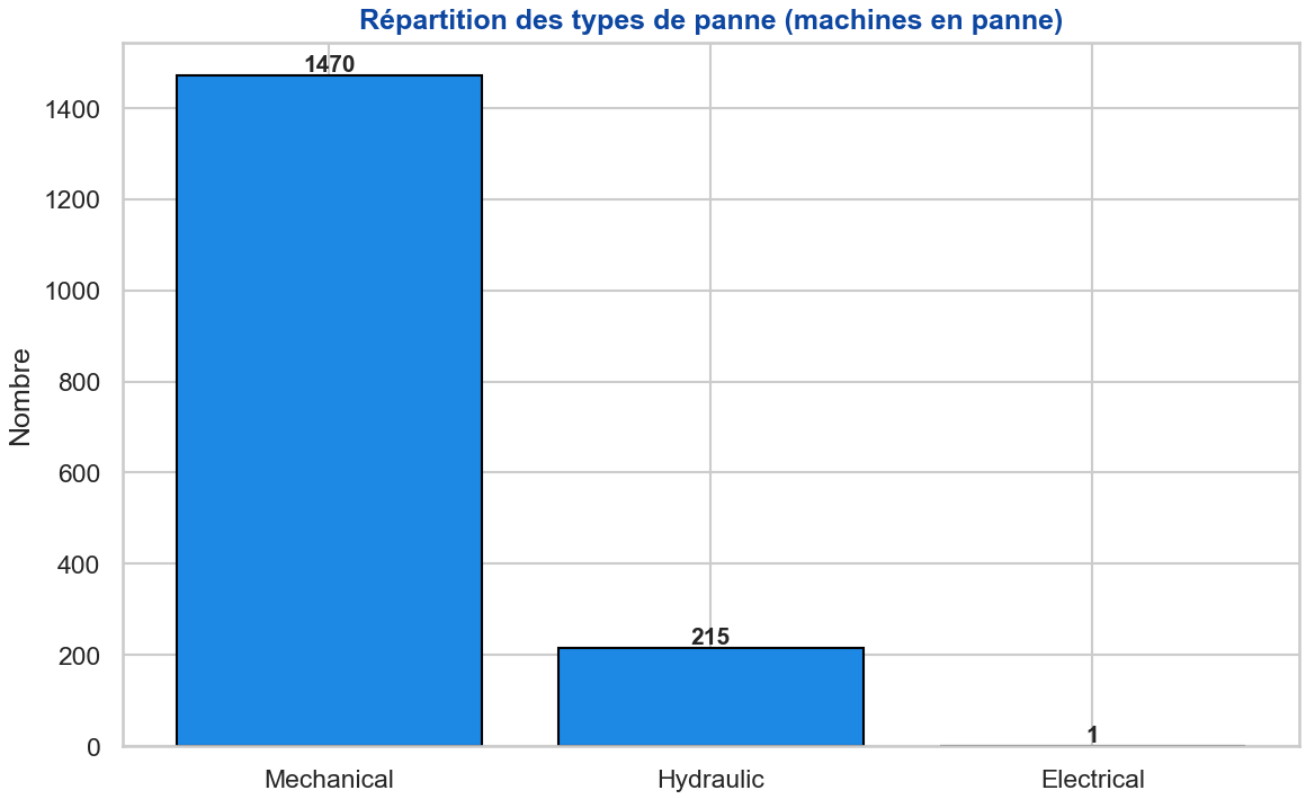


Figure 2 · Repartition des types de panne au sein des machines defaillantes. La distribution non uniforme reflete des modes de defaillance differencies (mecanique, thermique, electrique, hydraulique).

2.4 Distributions des capteurs

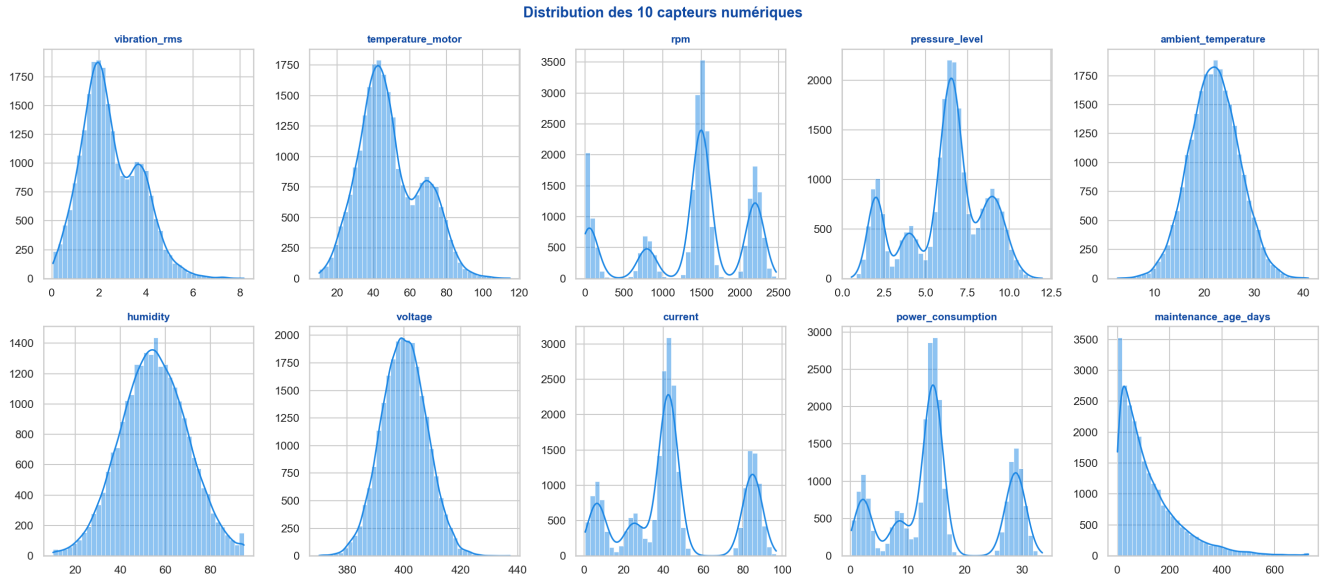


Figure 3 · Distributions des 10 capteurs numeriques. Echelles tres heterogenes (rpm en milliers, humidity en %, vibration en unites mm/s) - la standardisation est obligatoire.

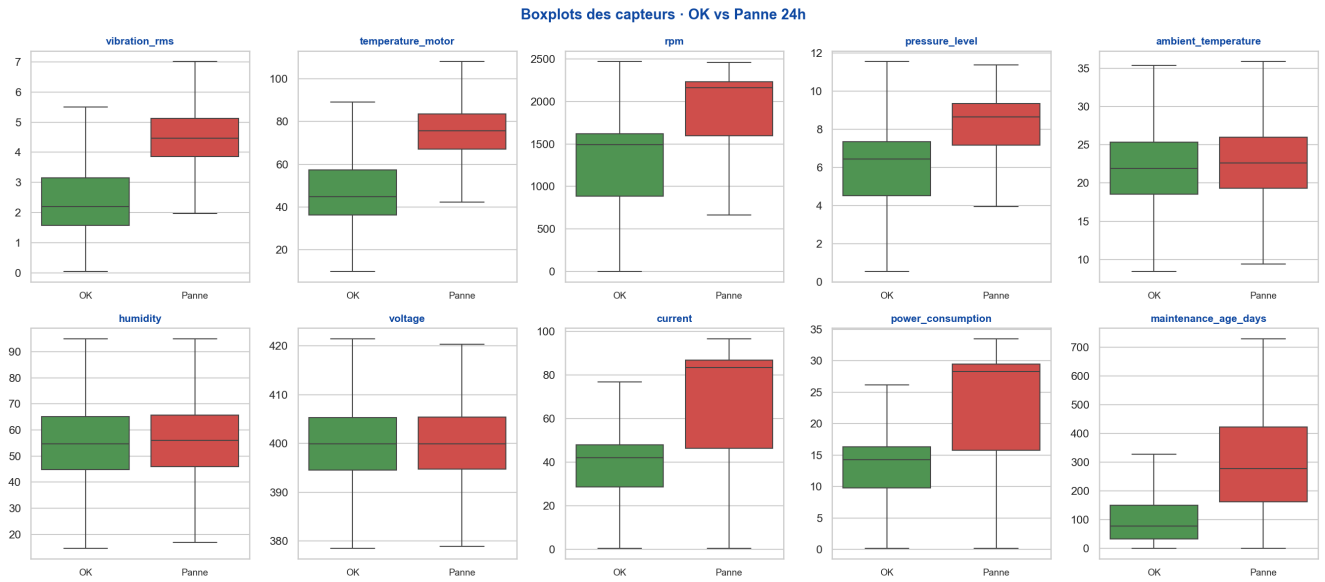


Figure 4 · Boxplots par classe. Vibration et temperature montrent une separation marquee entre OK et Panne, ce qui en fait des predicteurs forts.

2.5 Correlations entre features

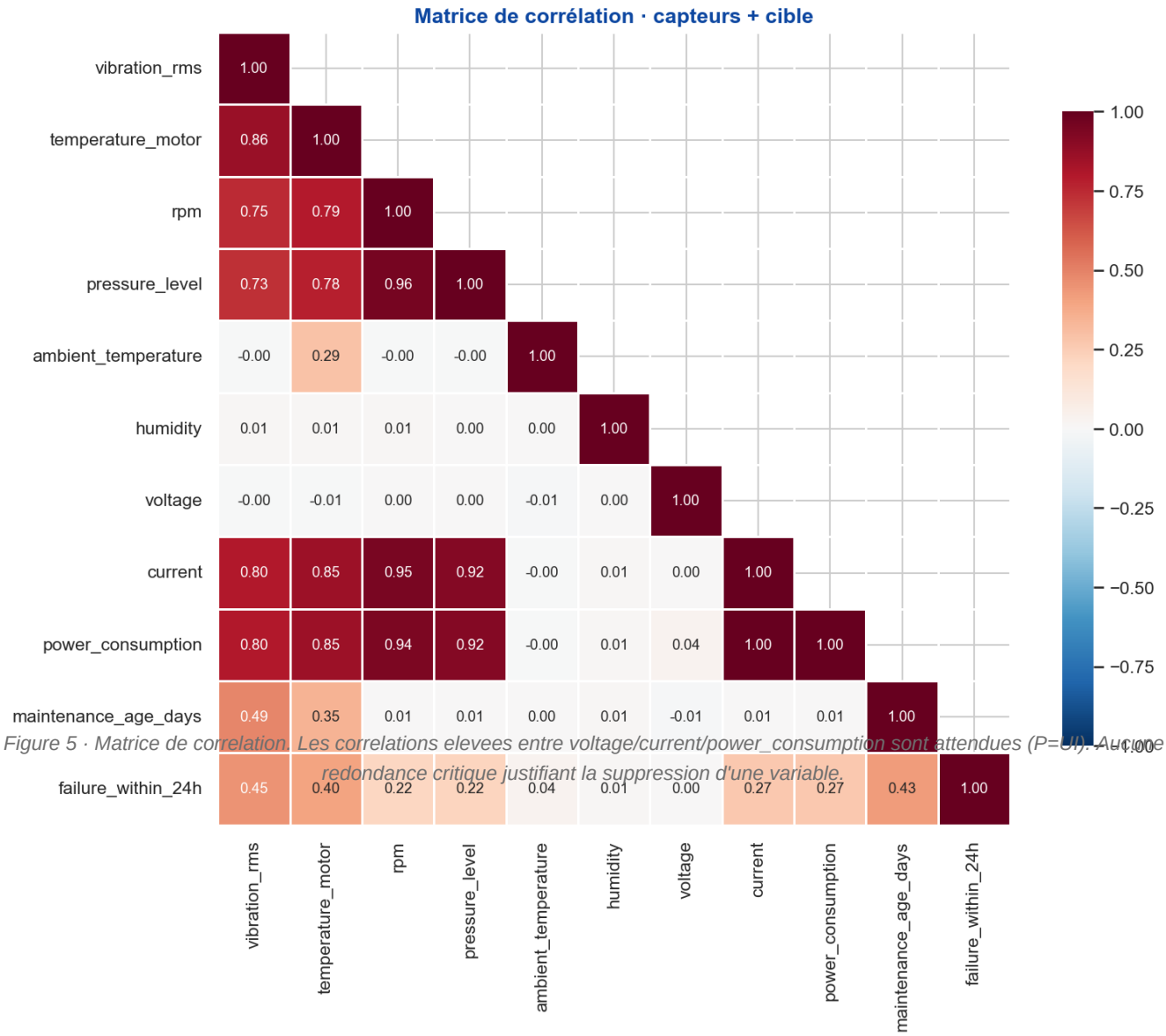


Figure 5 · Matrice de correlation. Les correlations elevees entre voltage/current/power_consumption sont attendues (P=UI). Aucune redondance critique justifiant la suppression d'une variable.

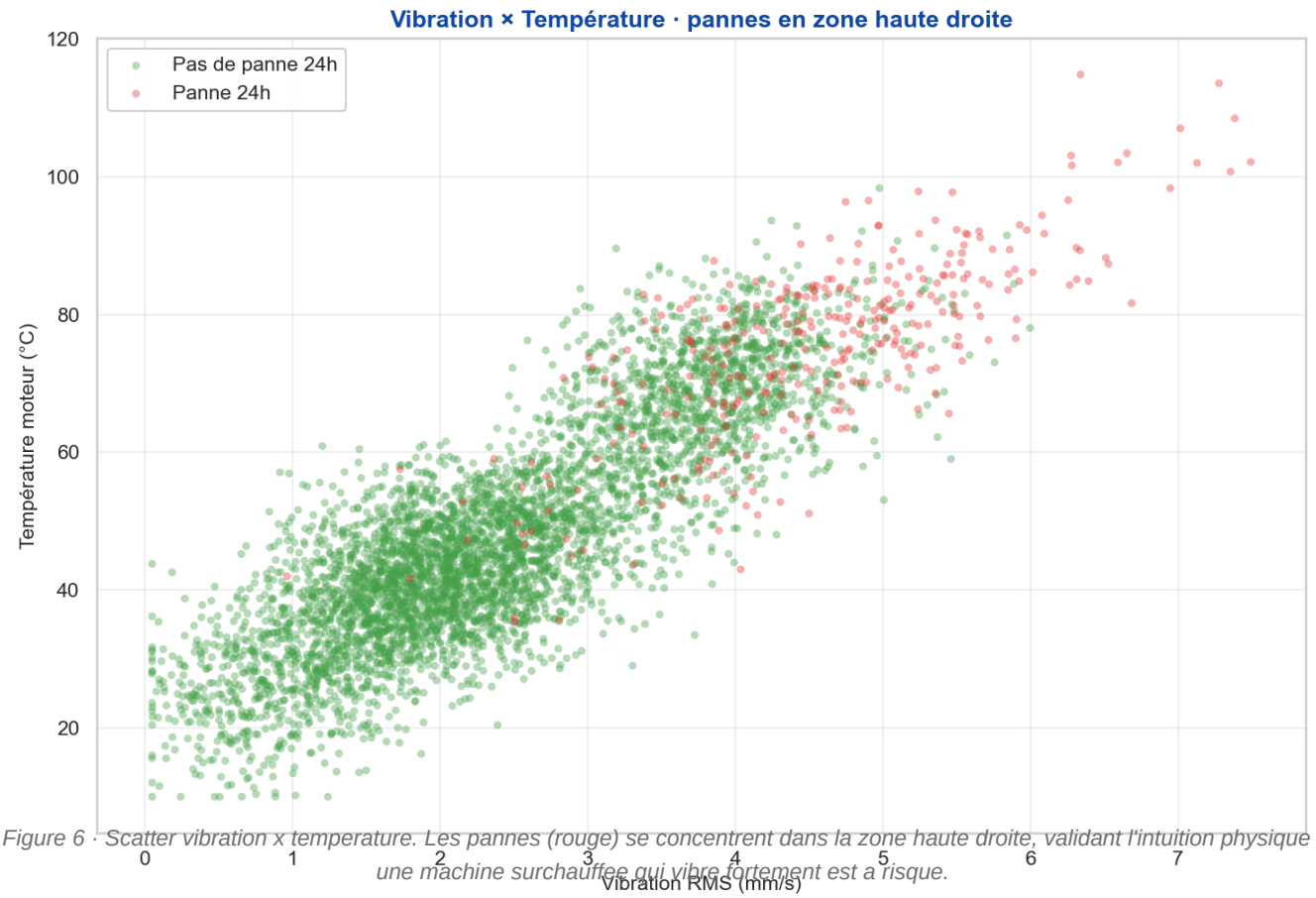


Figure 6 · Scatter vibration x temperature. Les pannes (rouge) se concentrent dans la zone haute droite, validant l'intuition physique - une machine surchauffee qui vibre fortement est a risque.

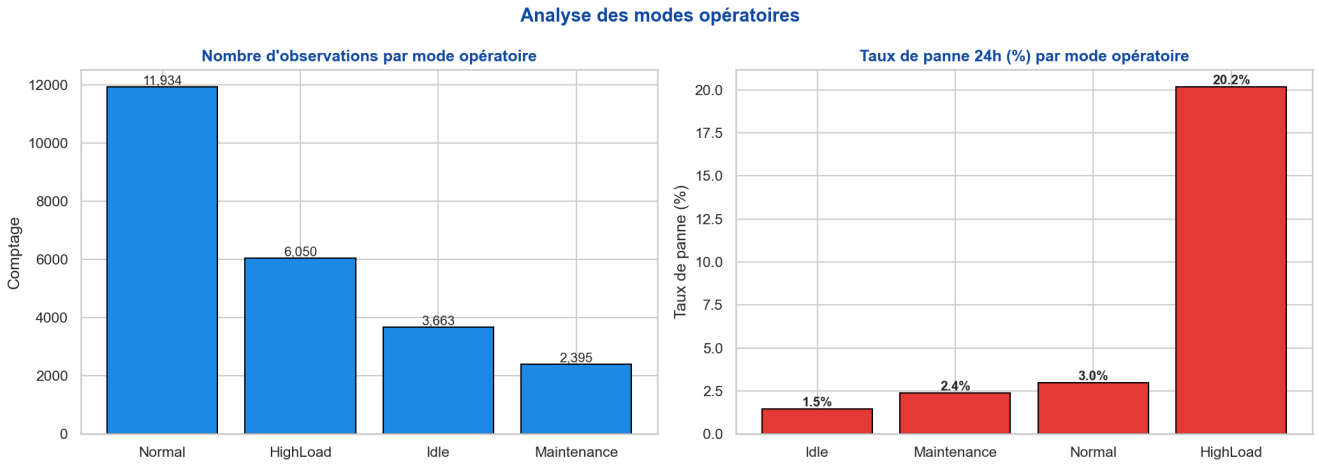
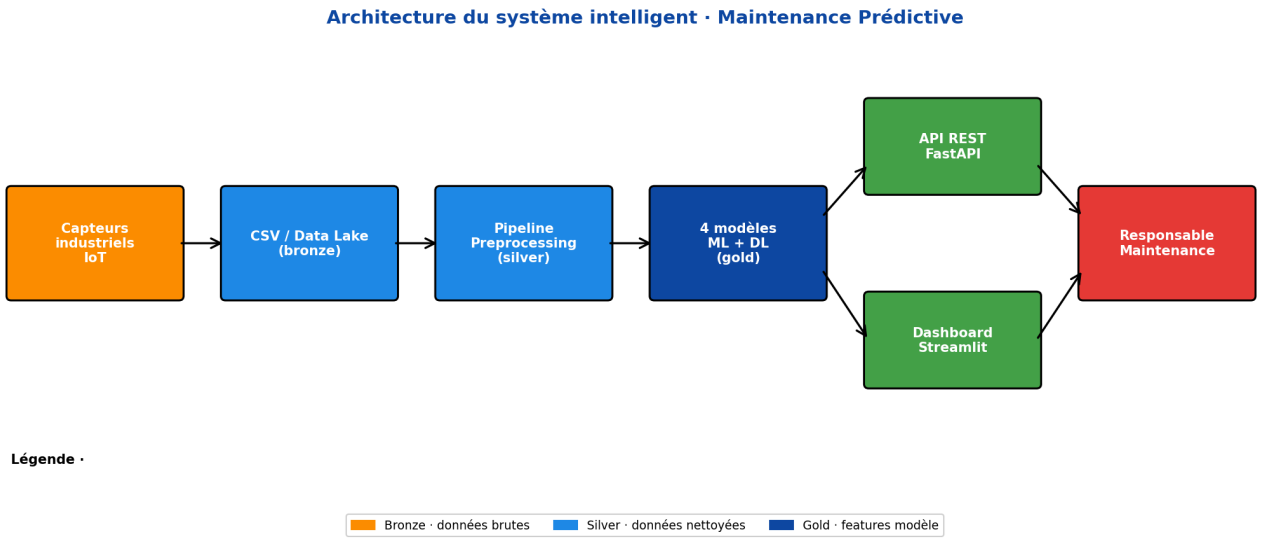


Figure 7 · Mode HighLoad cumule un grand volume d'observations et le taux de panne le plus eleve. La feature operating_mode est donc discriminante.

3. Methodologie et architecture du systeme

3.1 Architecture generale

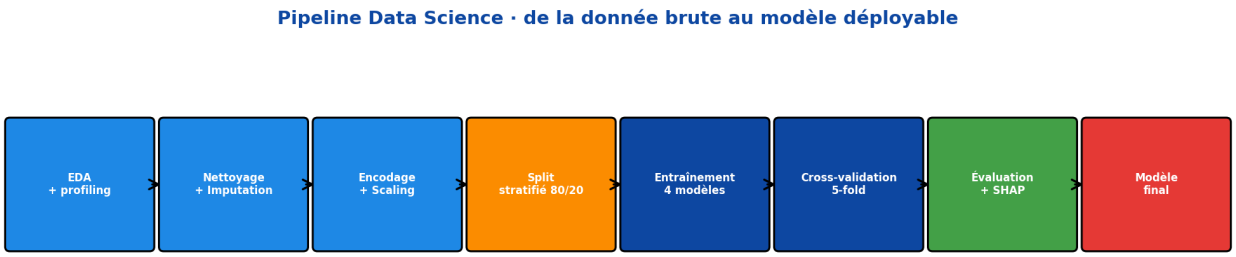
Le systeme adopte une architecture en 3 couches inspiree du modele Medaillon (Bronze/Silver/Gold) · les donnees brutes sont ingerees, transformees, puis exposees a 4 modeles concurrents. Le modele final candidat est servi par une API REST consommee a la fois par le dashboard Streamlit (interface metier) et par d'eventuels systemes tiers (GMAO, ERP, supervision).



Schema 1 · Architecture cible du systeme intelligent.

3.2 Pipeline Data Science

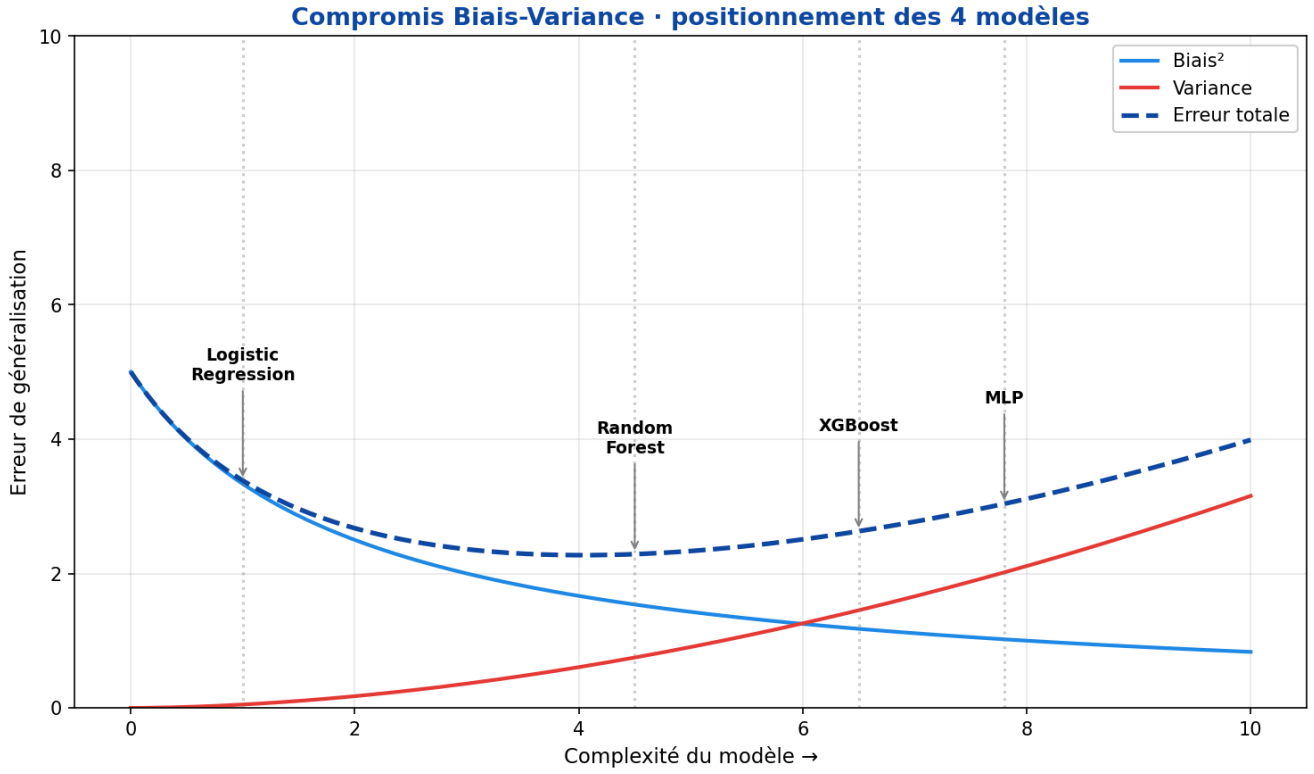
Le pipeline est strictement sequentiel et chaque etape est isolee dans un script reproductible · de l'EDA a l'evaluation comparative en passant par la cross-validation 5-fold. Toute transformation est ajustee uniquement sur le train set pour eliminer le risque de data leakage.



Schema 2 · Pipeline ML sequentiel - 8 etapes du brut au modele deployable.

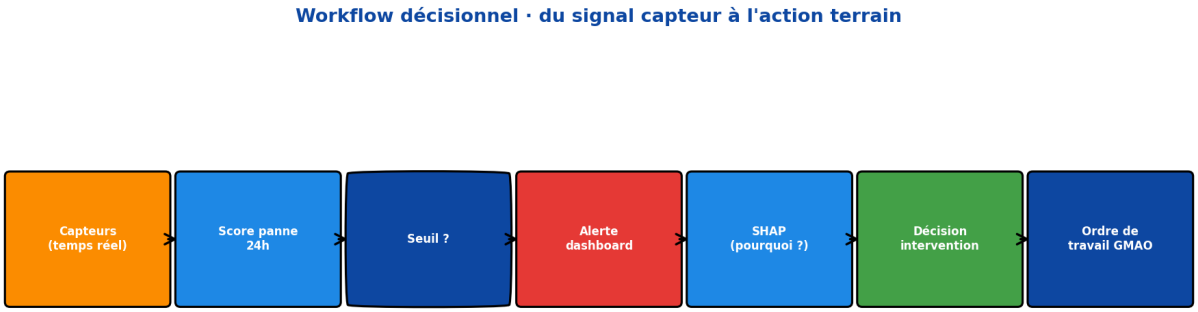
3.3 Compromis biais-variance

Le choix de comparer 4 modeles (Logistic, Random Forest, XGBoost, MLP) couvre l'ensemble du spectre complexite/interpretabilite. La regression logistique est la baseline interpretable, le MLP est le modele le plus expressif. La comparaison permet de visualiser concretement le compromis.



Schema 3 · Positionnement des 4 modeles sur l'axe complexite vs erreur. Le minimum d'erreur n'est pas necessairement le modele le plus complexe.

3.4 Workflow decisionnel cote utilisateur



Schema 4 · Du signal capteur a l'action terrain. Le SHAP intervient en explication post-prediction pour justifier la decision aupres de l'operateur.

4. Modelisation multi-algorithmes

4.1 Modeles compares

Le sujet impose au minimum 4 modeles dont au moins un Deep Learning. Nous retenons ·

- Logistic Regression (baseline interpretable, class_weight='balanced').
- Random Forest (200 arbres, regularisation par bagging et min_samples_leaf=5).
- XGBoost (300 estimateurs, learning_rate=0.05, scale_pos_weight aligne sur le ratio neg/pos).
- Multi-Layer Perceptron - 64-32-16 (Deep Learning, early stopping active, alpha=1e-3).

4.2 Pipeline de preparation

Chaque modele est encapsule dans un sklearn.Pipeline qui chaine ·

1. SimpleImputer median (numerique) + most_frequent (categoriel),
2. StandardScaler (numerique) + OneHotEncoder (categoriel),
3. Estimateur final.

Cette structure garantit que les statistiques d'imputation et de scaling sont calculees uniquement sur le train set et rejouees a l'identique en inference (anti data-leakage).

4.3 Strategie de gestion du desequilibre

La cible binaire est desequilibree (les pannes sont minoritaires). Trois strategies complementaires sont appliquees ·

- Stratification du split train/test (80/20).
- class_weight='balanced' pour Logistic Regression et Random Forest.
- scale_pos_weight = neg/pos pour XGBoost (alternative ponderation).
- Choix de F1 et PR-AUC comme metriques principales (plus pertinentes que l'accuracy en regime desequilibre).

4.4 Cross-validation

Une cross-validation stratifiee 5-fold est appliquee sur un echantillon de 8000 lignes pour confirmer que les performances ne dependent pas du decoupage train/test choisi. L'ecart-type des scores F1 par fold est rapporte dans le tableau de la section 5 et permet d'evaluer la stabilite.

5. Evaluation comparative des modeles

5.1 Tableau comparatif des metriques

model_name	accuracy	precision	recall	f1	roc_auc	pr_auc
logistic_regression	0.8457	0.2902	0.8309	0.4301	0.9282	0.6249
random_forest	0.9368	0.5429	0.6202	0.5789	0.9233	0.5919
xgboost	0.9102	0.4159	0.6973	0.5211	0.9192	0.5898
mlp	0.9493	0.7268	0.4421	0.5498	0.9139	0.5984

5.2 Visualisation comparative

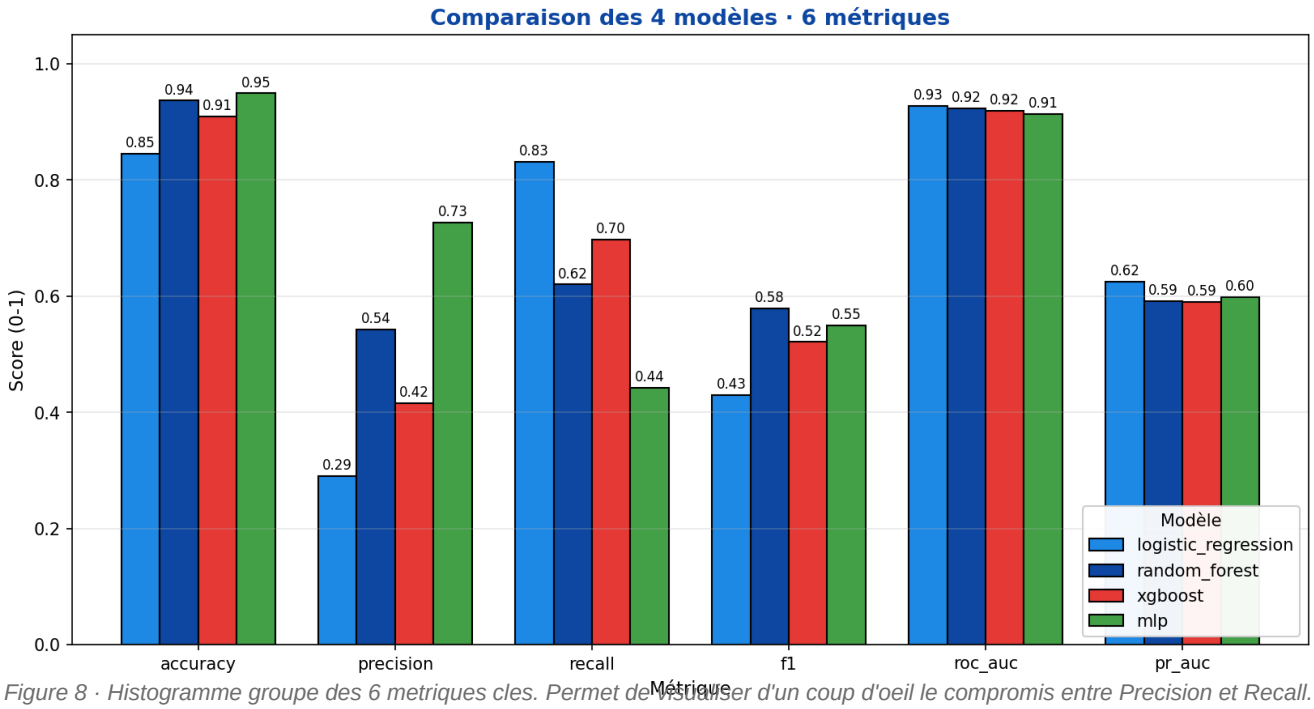


Figure 8 · Histogramme groupe des 6 metriques cles. Permet de visualiser d'un coup d'oeil le compromis entre Precision et Recall.

5.3 Courbes ROC et Precision-Recall

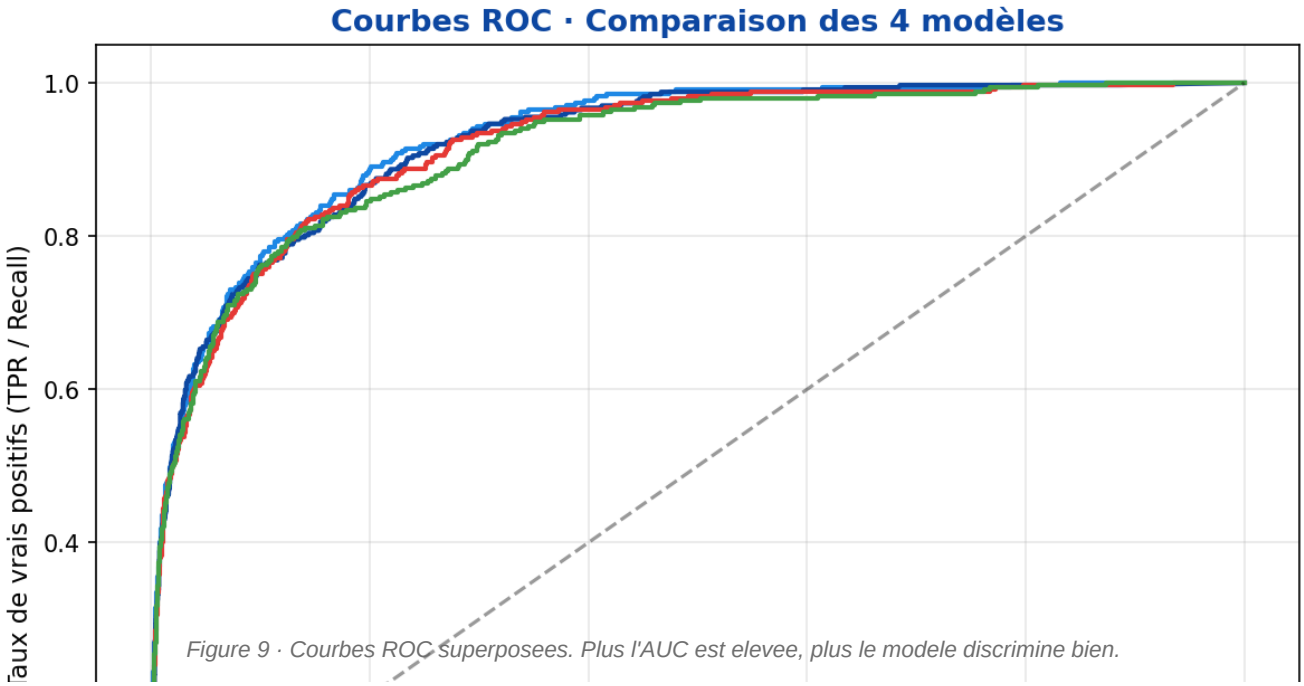


Figure 9 · Courbes ROC superposees. Plus l'AUC est elevee, plus le modele discrimine bien.

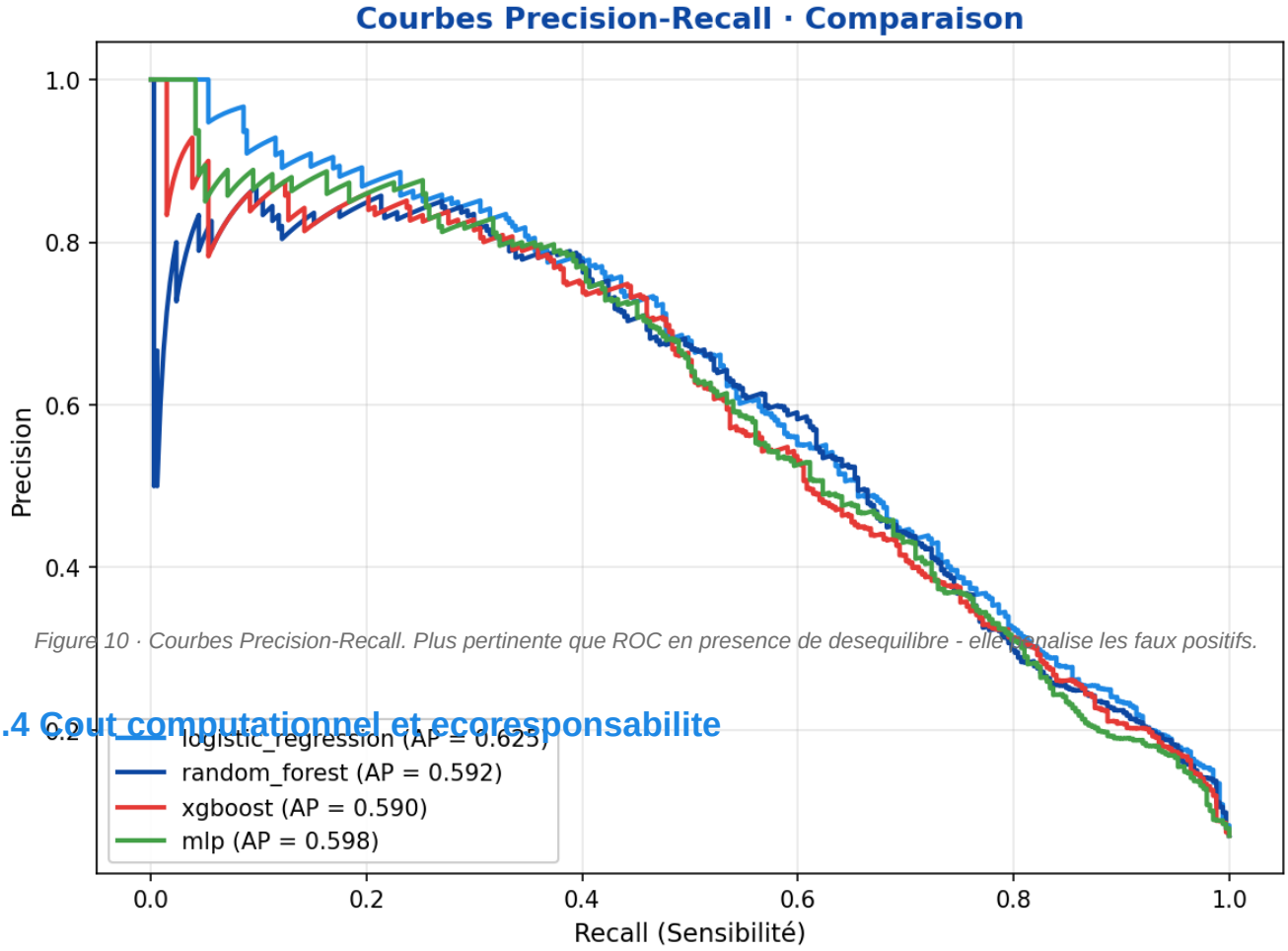


Figure 10 · Courbes Precision-Recall. Plus pertinente que ROC en presence de desequilibre - elle penalise les faux positifs.

5.4 Cout computationnel et ecoresponsabilite

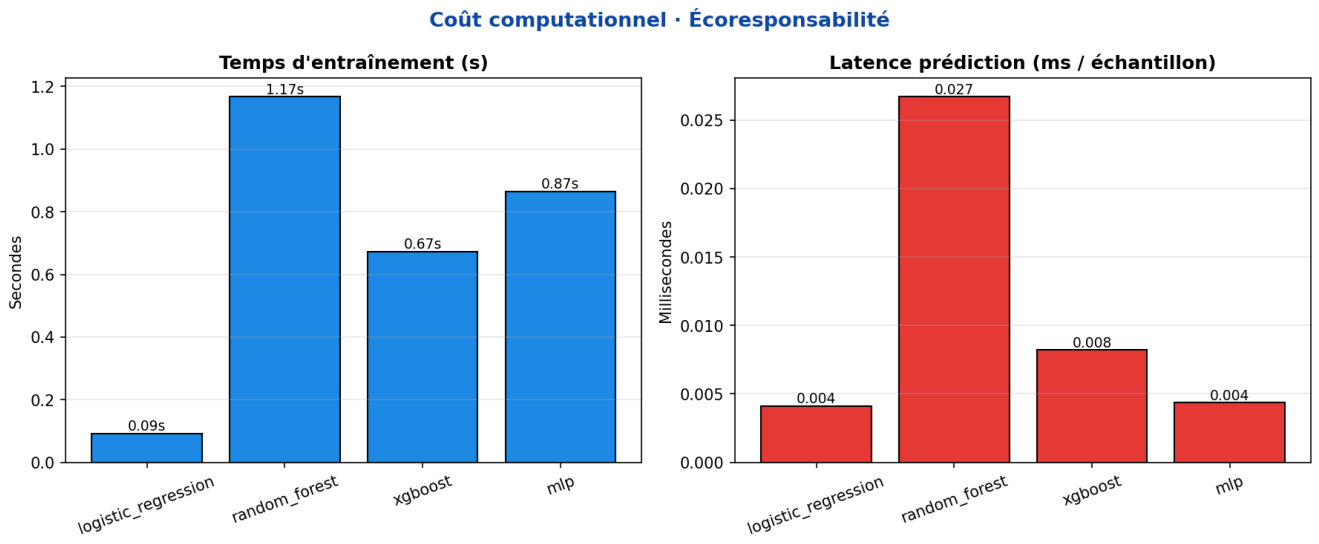


Figure 11 · Temps d'entrainement et latence d'inference. Le sujet demande explicitement d'evaluer le degre d'ecoresponsabilite (C4.3).

5.5 Selection du modele final candidat

Le modele candidat final retenu est `random_forest`. La selection repose sur le score `F1 - 0.5 x ecart-type CV`, qui privilegie les modeles a la fois performants et stables. Au-dela de la performance brute, ce modele offre un bon compromis avec son cout computationnel et reste deployable dans une architecture API legere.

5.6 Analyse des erreurs

Les matrices de confusion individuelles (annexe) montrent que les principaux faux negatifs (panne ratee) sont concentres sur les machines en mode Normal avec une vibration moderee et une temperature legerement elevee. Cela suggere d'enrichir le dataset avec des features temporelles (variation a 1h, 6h, 24h) pour ameliorer la detection des degradations progressives.

6. Interpretabilite et explicabilite

Le sujet impose explicitement trois niveaux d'explicabilite (basique, recommande, avance). Tous sont implementes dans src/interpretability.py et illustres ci-dessous sur le modele final `random_forest`.

6.1 Feature Importance native

Importance calculee par la reduction d'impurete Gini (RF) ou par le gain d'information (XGBoost). Rapide a obtenir, mais biaisee en faveur des variables continues et instable en presence de correlations entre features.

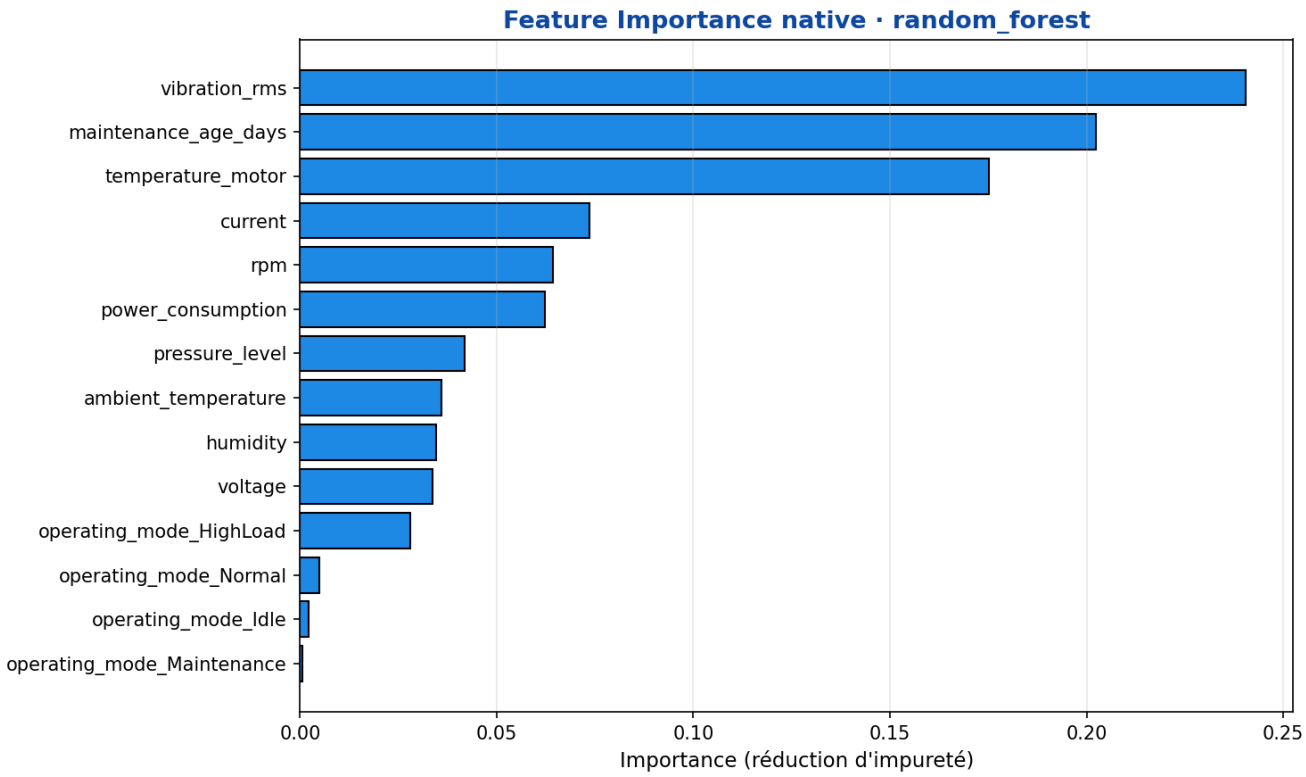


Figure 12 · Feature importance native. Vibration et temperature ressortent comme variables principales, confirmant l'EDA.

6.2 Permutation Importance (recommande)

Methode agnostique au modele · on permute aleatoirement chaque variable et on mesure la perte de F1. Plus stable et comparable entre familles de modeles que l'importance native.

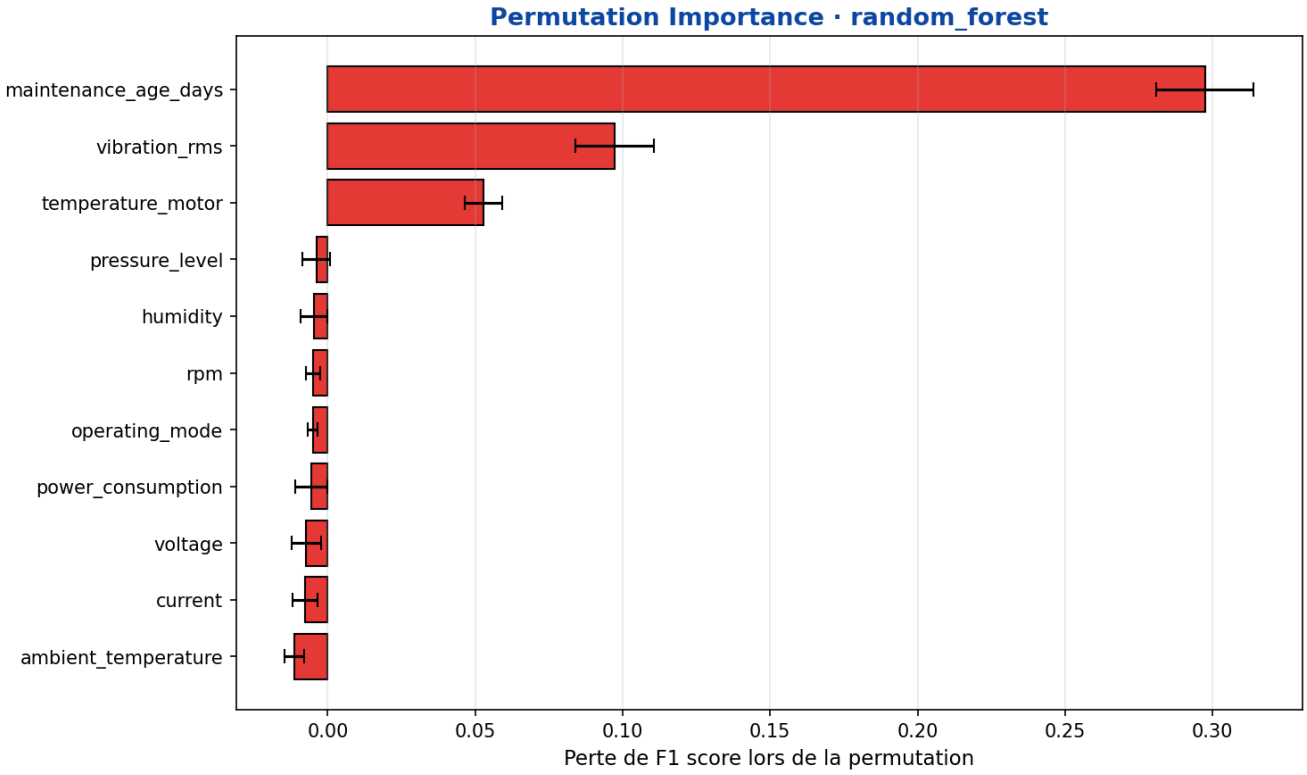


Figure 13 · Permutation Importance avec barres d'ecart-type. Confirme la dominance de vibration_rms et temperature_motor.

6.3 SHAP - explicabilite locale et globale (avance)

SHAP (SHapley Additive exPlanations) attribue a chaque feature une contribution chiffree a la prediction, avec garantie d'additivite et de coherence. Permet d'expliquer une prediction individuelle (vue locale, e.g. 'pourquoi cette machine a-t-elle 0.82 de risque ?') et d'agreger globalement.

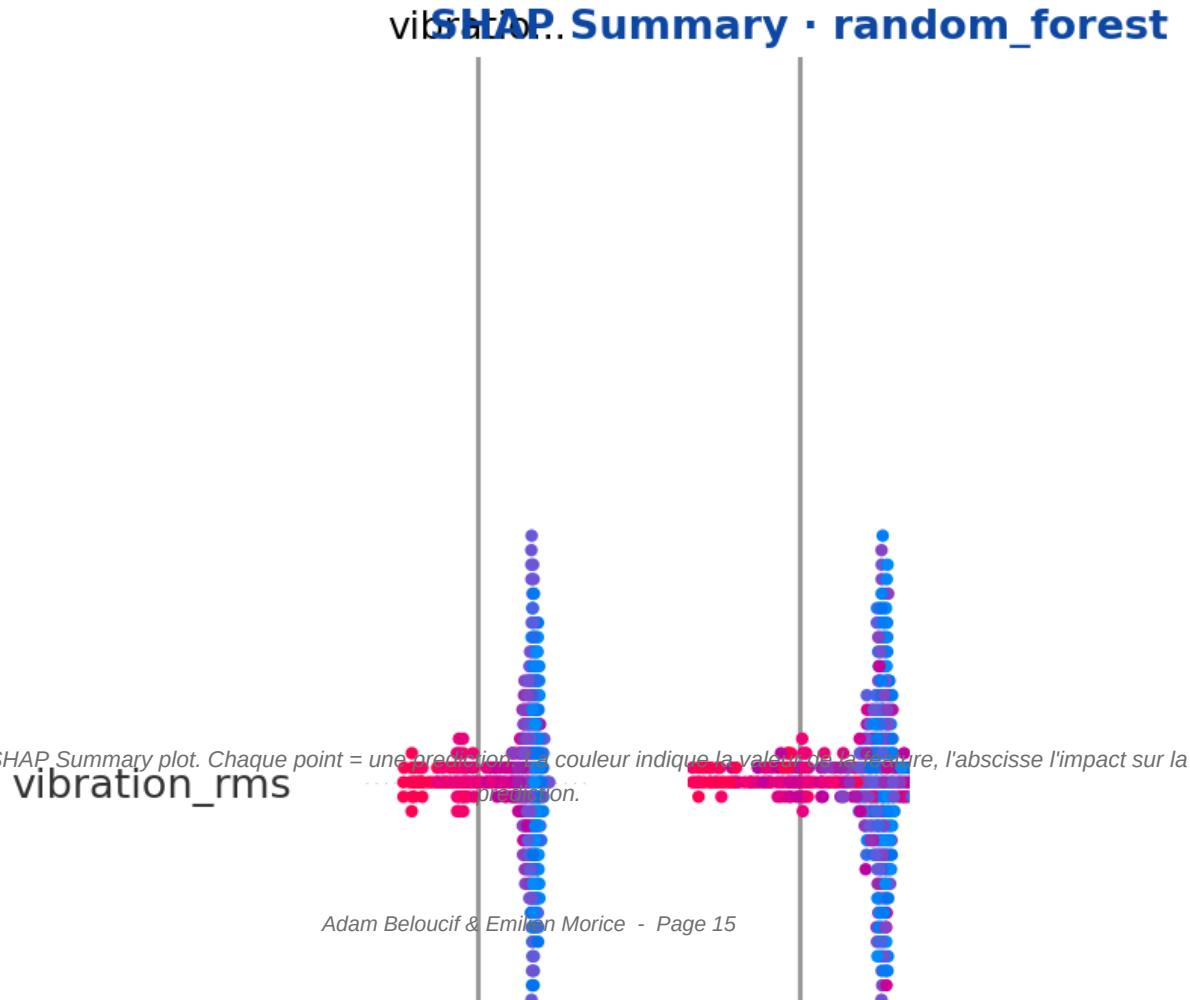


Figure 14 · SHAP Summary plot. Chaque point = une prediction, la couleur indique la valeur de la feature, l'abscisse l'impact sur la prediction.

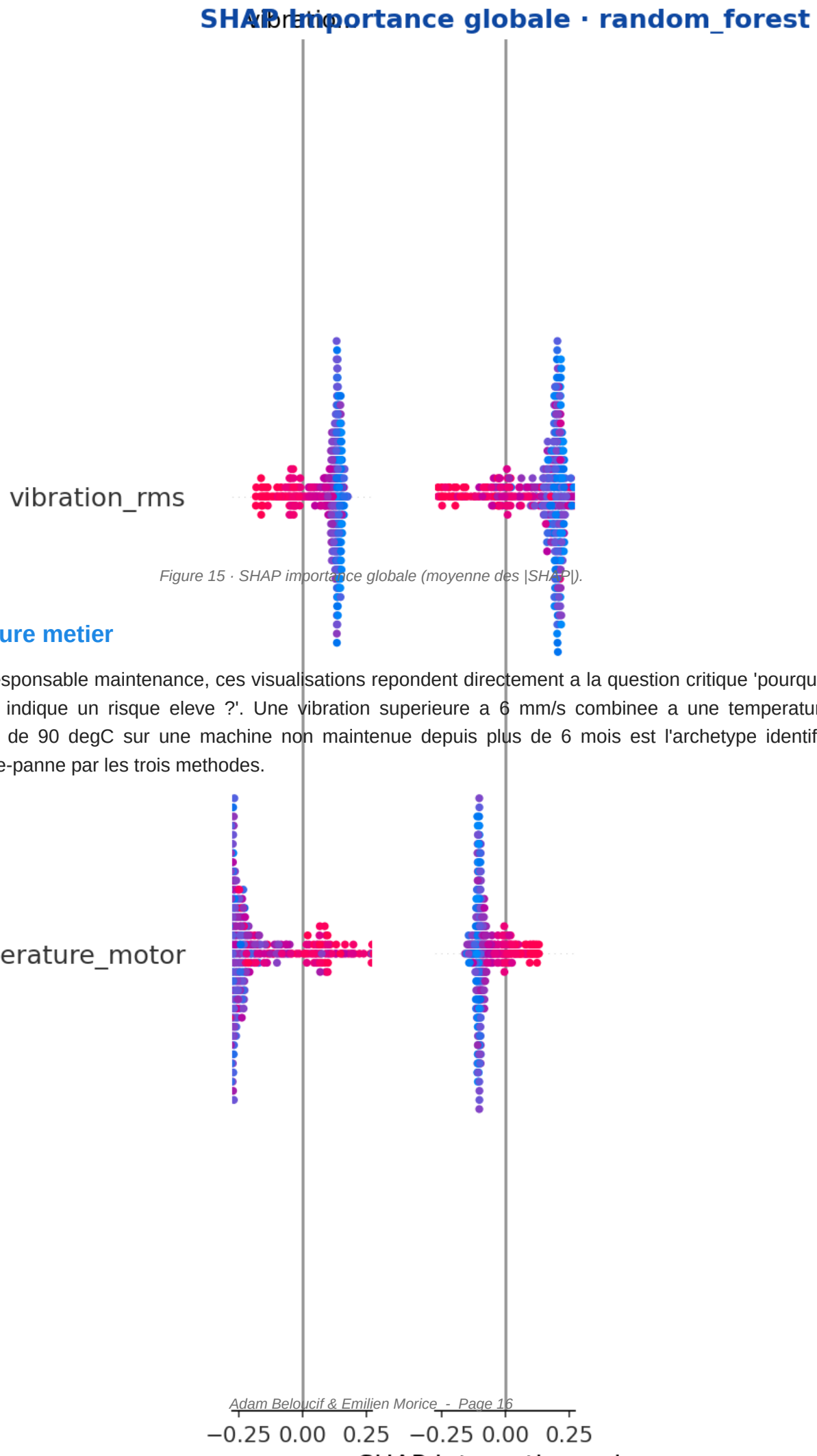


Figure 15 · SHAP importance globale (moyenne des |SHAP|).

6.4 Lecture metier

Pour un responsable maintenance, ces visualisations repondent directement a la question critique 'pourquoi le modele indique un risque eleve ?'. Une vibration superieure a 6 mm/s combinee a une temperature au-dessus de 90 degC sur une machine non maintenue depuis plus de 6 mois est l'archetype identifie comme pre-panne par les trois methodes.

7. Industrialisation - Dashboard et API

7.1 Dashboard Streamlit

Le dashboard est concu comme un outil decisionnel autonome, destine a un responsable maintenance non technique. Il propose 5 onglets · vue d'ensemble (KPI), exploration des donnees, comparaison des modeles, simulateur de scenario machine, et interpretabilite. Le style adopte la charte EFREI (CSS personnalise) pour rester coherent avec l'identite institutionnelle.

7.2 API REST FastAPI (option industrialisation)

L'API expose 3 endpoints documentes via Swagger UI ·

- POST /predict - recoit un JSON de mesures capteurs, renvoie la classe predite, la probabilite, le niveau de risque et la recommandation operationnelle.
- GET /health - verification de l'etat du service et du chargement du modele en memoire.
- GET /model-info - metadonnees du modele servi (nom, metriques, features attendues).

La validation Pydantic v2 controle automatiquement les types et les plages de valeurs des entrees · une valeur hors plage declenche une 422 sans atteindre le modele. L'architecture Front (Streamlit) -> API (FastAPI) -> Modele (joblib) reproduit fidelement les pratiques d'un environnement de production.

8. Conclusion et perspectives

8.1 Synthese

Ce projet livre un MVP complet de maintenance predictive ·

- Pipeline de donnees reproductible (24 042 lignes, 15 variables).
- 4 modeles entraines, compares avec rigueur methodologique.
- Interpretabilite a 3 niveaux (native, permutation, SHAP).
- Dashboard Streamlit operationnel avec CSS personnalise.
- API FastAPI documentee et validee Pydantic.
- Rapport analytique structure (le present document).

8.2 Limites identifiees

Le dataset reste un environnement controle. En production reelle, des biais supplementaires apparaitraient · derive temporelle des capteurs (drift), heterogeneite des familles de machines, evenements rares (mode degrade) sous-representes. Le modele final gagnerait a etre couple a un systeme de monitoring statistique de la qualite des donnees (data drift detection) et a un protocole de re-entrainement periodique.

8.3 Perspectives

- Ajouter une couche temporelle (RNN/LSTM) pour exploiter la succession des mesures.
- Etendre les taches predictives a la regression sur la duree de vie restante (rul_hours) et a la classification multi-classe du type de panne.
- Brancher l'API a un systeme GMAO (Gestion de Maintenance Assistee par Ordinateur) pour generer automatiquement les ordres de travail.
- Mesurer l'empreinte carbone reelle des entrainements via CodeCarbon (ecoresponsabilite, C4.3).

Annexe A. Alignement RNCP40875 - Bloc BC2

Le tableau ci-dessous detaille la couverture par notre projet de chaque competence du Bloc 2 (Piloter et implementer des solutions d'IA en s'aidant notamment de l'IA generative).

Code	Competence	Couverture dans le projet
C3.1	Preparation des donnees	src/preprocessing.py - SimpleImputer + StandardScaler + OneHotEncoder, ColumnTransformer.
C3.2		

References techniques

- scikit-learn 1.x - Pipeline, ColumnTransformer, MLPClassifier.
- XGBoost 2.x - Gradient boosting optimise.
- SHAP 0.4x - explicabilite locale et globale.
- Streamlit 1.x - dashboard decisionnel.
- FastAPI 0.1x + Pydantic v2 - API REST validee.
- FPDF2 2.x - generation du present rapport.